

DBS/DAS possible future developments

Hybrid workflow management systems: CMS+DIRAC(X) or
CMS+PAnda

Todor Ivanov

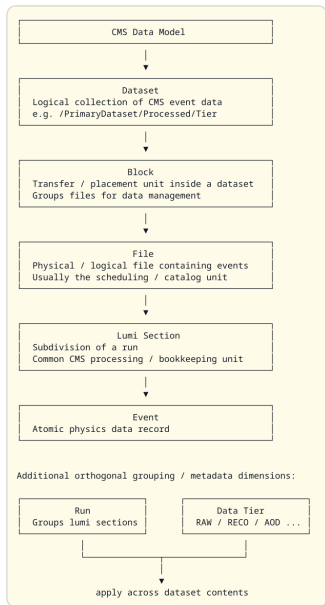
University of Notre Dame

18-03-2026

Main assumptions

Main assumptions:

- ▶ The CMS Data model does not change ^a
- ▶ We implement the new system and do not change patterns on how we use the DATA Book keeping system:
 - ▶ DBS - data discovery and resolution
 - ▶ Rucio - data movement



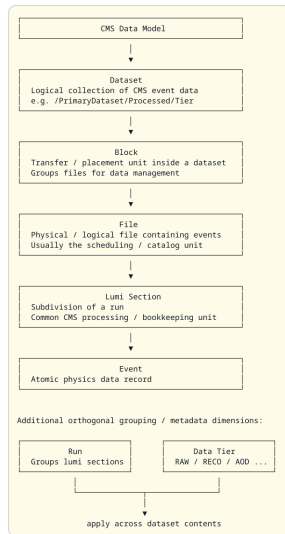
^a <https://github.com/todor-ivanov/CMSDiracAux>

Main assumptions

```
if x:
    print(
        "we_are_here"
    )
```

- ▶ The CMS Data model does not change ^a
- ▶ We implement the new system and do not change patterns on how we use the DATA Book keeping system:
 - ▶ DBS - data discovery and resolution
 - ▶ Rucio - data movement

^a <https://github.com/todor-ivanov/CMSDiracAux>



Test column

Test Column

- ▶ The CMS Data model does not change¹
- ▶ We implement the new system and do not change patterns on how we use the DATA Book keeping system:
 - ▶ DBS - data discovery and resolution
 - ▶ Rucio - data movement

A	B
42	17
89	63
25	74

¹<https://github.com/todor-ivanov/CMSDiracAux>

First scenario

- ▶ Possible Places of interaction with DBS/DAS and eventual code changes (under diff conditions)
 - ▶ Read
 - ▶ CMS+DIRAC:²
 - ▶ READ interactions: - in case the interaction between the systems (the CMS data resolution) happens through the DIRAC native DAtaPlacement system (calls to Dirac Request manager + (or) Dirac File Catalog queries) - The code change should go inside the DIRAC system (not a good idea), but we do not know how extendable is this part of the DIRAC system (could it use a plugin based mechanism for adding extensions to the system similarly to the Dirac Transformations system) - The new component becomes native to the DIRAC system - the system does not evolve any more(may be impossible). Second obstacle - this approach means it would/should communicate with DBS/DAS through its protocol stack which is not HTTP API oriented (RPC calls etc.). - implies huge change on the DAS/DBS side
 - ▶ WRITE interactions:
 - ▶ At job batch completion - the interaction should be initiated from which ever BookKeepoing system is serving the purpose - for both DIRAC and DIRACx

Second scenario

Second scenario

Main assumption:

We evolve DAS to a full fledged data aggregation and access service

- ▶ The CMS Data model does not change
- ▶ We implement the new system and do not change patterns on how we use the DATA Book keeping system:
 - ▶ DBS - data discovery and resolution
 - ▶ Rucio - data movement
- ▶ DAS is already quering DBS and Rucio and is capable of providing full concept of data structure&placement and already delivers reliable data discovery and resolution infrastructure in such approach
 - ▶ we will heavily rely on DAS for this service, on all levels of workload granulrity - which means we may start hitting it hard, so we may have to face scalability issues, but should not be unsolvable, since DAS is a thin layer infornt of DBS
 - ▶ we will still use DBS for metadata records as before, which means the only direct queries to DBS would be write on block requests

Third scenario

Move entire DBS content to Rucio use the Rucio's capabilities to handle metadata

- ▶ the same complexity introduction problem as the one which appears inside the Internal Representation layer for CMSTDiracAux
 - ▶ abstractions mismatch
 - ▶ GRanularity and Data storage levels mismatch - block/dataset misalignment yet again
 - ▶ Possible match is the data contents - physics objects should not be too different , but the question remains how capable is Rucio for handling metadata at levels going down to luminosity sections and events
 - ▶ Possible scalability issues
- ▶ the good part - the whole project immediately transforms into a single OPS action - pouring the metadata contents inside Rucio

BUT the long term operation remains

Conclusions

MAIN MESSAGE: Provided the CMS data model does not change and the atomic unit still stays a luminosity section, no structural changes to the Data bookkeeping services are to be needed other than possible API interfaces evolution. It is not the job granularity and the way of we split the load over the data which creates the pressure on DBS, It is the system level at which we initiate the interaction with DBS, because at different levels of the system we we handle different levels of workload abstractions, which represent different workflow granularity and hence affect the frequency of querying and the amount of data transferred from the metadata system.